



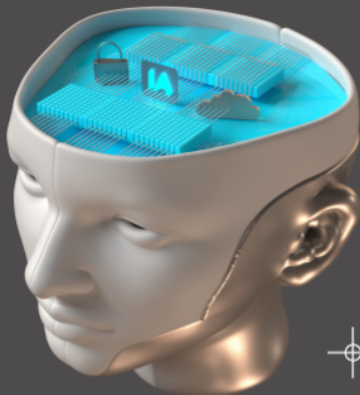
UTEC Posgrado

DEEP LEARNING GRADIENT DESCENT



Motivación

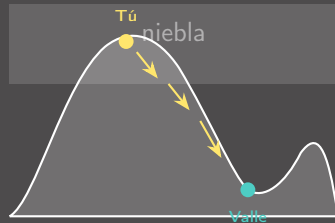
Contexto e intuición del Gradient Descent



Imagina que estás perdido en una montaña con niebla

Objetivo: Llegar al punto más bajo del valle.

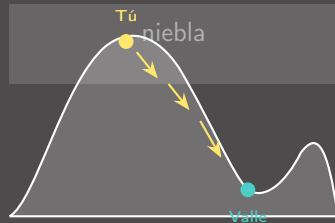
- ▶ No puedes ver el valle completo (hay niebla)



Imagina que estás perdido en una montaña con niebla

Objetivo: Llegar al punto más bajo del valle.

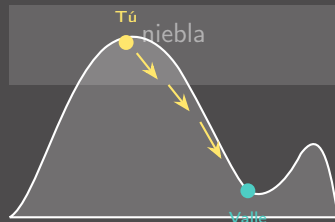
- ▶ No puedes ver el valle completo (hay niebla)
- ▶ Solo puedes sentir la **pendiente** bajo tus pies



Imagina que estás perdido en una montaña con niebla

Objetivo: Llegar al punto más bajo del valle.

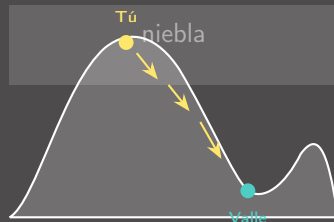
- ▶ No puedes ver el valle completo (hay niebla)
- ▶ Solo puedes sentir la **pendiente** bajo tus pies
- ▶ Estrategia: dar pasos siempre **cuesta abajo**



Imagina que estás perdido en una montaña con niebla

Objetivo: Llegar al punto más bajo del valle.

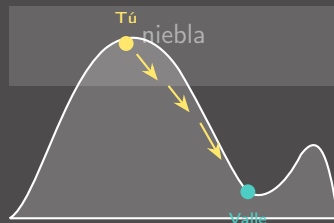
- ▶ No puedes ver el valle completo (hay niebla)
- ▶ Solo puedes sentir la **pendiente** bajo tus pies
- ▶ Estrategia: dar pasos siempre **cuesta abajo**
- ▶ Poco a poco llegas al punto más bajo



Imagina que estás perdido en una montaña con niebla

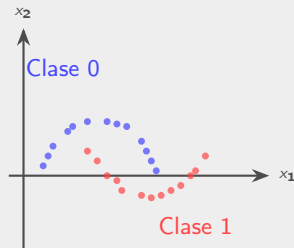
Objetivo: Llegar al punto más bajo del valle.

- ▶ No puedes ver el valle completo (hay niebla)
 - ▶ Solo puedes sentir la **pendiente** bajo tus pies
 - ▶ Estrategia: dar pasos siempre **cuesta abajo**
 - ▶ Poco a poco llegas al punto más bajo
- Eso es **Gradient Descent**: bajar por la pendiente de una función paso a paso.



Nuestro problema: clasificar datos no linealmente separables

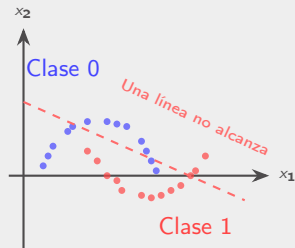
Contexto: Tenemos 400 puntos en forma de lunas (moons) con dos clases. Una línea recta **no puede** separarlos.



Nuestro problema: clasificar datos no linealmente separables

Contexto: Tenemos 400 puntos en forma de lunas (moons) con dos clases. Una línea recta **no puede** separarlos.

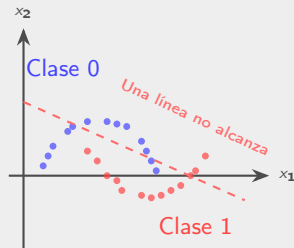
- Necesitamos una **red neuronal** con capa oculta



Nuestro problema: clasificar datos no linealmente separables

Contexto: Tenemos 400 puntos en forma de lunas (moons) con dos clases. Una línea recta **no puede** separarlos.

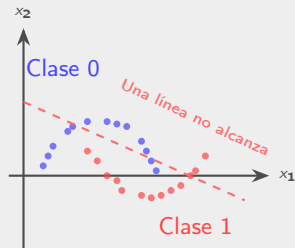
- Necesitamos una **red neuronal** con capa oculta
- La red tiene **pesos** (W_1, W_2) y **sesgos** (b_1, b_2)



Nuestro problema: clasificar datos no linealmente separables

Contexto: Tenemos 400 puntos en forma de lunas (moons) con dos clases. Una línea recta **no puede** separarlos.

- ▶ Necesitamos una **red neuronal** con capa oculta
- ▶ La red tiene **pesos** (W_1, W_2) y **sesgos** (b_1, b_2)
- ▶ ¿Cómo encuentra la red los mejores pesos?

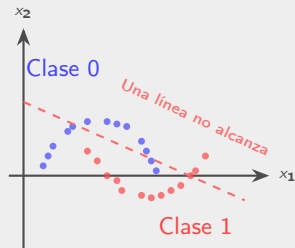


Nuestro problema: clasificar datos no linealmente separables

Contexto: Tenemos 400 puntos en forma de lunas (moons) con dos clases. Una línea recta **no puede** separarlos.

- ▶ Necesitamos una **red neuronal** con capa oculta
- ▶ La red tiene **pesos** (W_1, W_2) y **sesgos** (b_1, b_2)
- ▶ ¿Cómo encuentra la red los mejores pesos?

⇒ **Gradient Descent** ajusta los pesos iterativamente para minimizar el error de clasificación.



¿Qué es la función de costo?

Es una medida numérica del **error** del modelo. En clasificación binaria usamos **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$



¿Qué es la función de costo?

Es una medida numérica del **error** del modelo. En clasificación binaria usamos **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

► $y^{(i)} \in \{0, 1\}$: la clase real del punto i



¿Qué es la función de costo?

Es una medida numérica del **error** del modelo. En clasificación binaria usamos **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- ▶ $y^{(i)} \in \{0, 1\}$: la clase real del punto i
- ▶ $\hat{y}^{(i)} \in (0, 1)$: la probabilidad predicha



¿Qué es la función de costo?

Es una medida numérica del **error** del modelo. En clasificación binaria usamos **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- ▶ $y^{(i)} \in \{0, 1\}$: la clase real del punto i
- ▶ $\hat{y}^{(i)} \in (0, 1)$: la probabilidad predicha
- ▶ Si $y = 1$ y $\hat{y} \approx 1 \rightarrow \mathcal{L} \approx 0$ (bien)
- ▶ Si $y = 1$ y $\hat{y} \approx 0 \rightarrow \mathcal{L}$ es grande (mal)

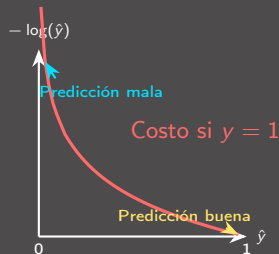


¿Qué es la función de costo?

Es una medida numérica del **error** del modelo. En clasificación binaria usamos **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- ▶ $y^{(i)} \in \{0, 1\}$: la clase real del punto i
- ▶ $\hat{y}^{(i)} \in (0, 1)$: la probabilidad predicha
- ▶ Si $y = 1$ y $\hat{y} \approx 1 \rightarrow \mathcal{L} \approx 0$ (bien)
- ▶ Si $y = 1$ y $\hat{y} \approx 0 \rightarrow \mathcal{L}$ es grande (mal)



Ejemplo: calculemos el costo con 3 pacientes

Contexto: Un modelo predice si un paciente tiene una enfermedad ($y = 1$) o no ($y = 0$).

Nota: El logaritmo usado es el **logaritmo natural** ($\ln$, base $e \approx 2,718$), estándar en Deep Learning.

Paciente	Real (y)	Pred. ($\hat{y}$)	Costo individual
Ana	1 (enferma)	0.9	$-\ln(0,9) = 0,105$
Bruno	0 (sano)	0.2	$-\ln(0,8) = 0,223$
Carla	1 (enferma)	0.3	$-\ln(0,3) = 1,204$



Ejemplo: calculemos el costo con 3 pacientes

Contexto: Un modelo predice si un paciente tiene una enfermedad ($y = 1$) o no ($y = 0$).

Nota: El logaritmo usado es el **logaritmo natural** ($\ln$, base $e \approx 2,718$), estándar en Deep Learning.

Paciente	Real (y)	Pred. ($\hat{y}$)	Costo individual
Ana	1 (enferma)	0.9	$-\ln(0,9) = 0,105$
Bruno	0 (sano)	0.2	$-\ln(0,8) = 0,223$
Carla	1 (enferma)	0.3	$-\ln(0,3) = 1,204$

Costo total:

$$\begin{aligned}
 \mathcal{L} &= -\frac{1}{3} \left[\underbrace{\ln(0,9)}_{\text{Ana: bien}} + \underbrace{\ln(0,8)}_{\text{Bruno: bien}} + \underbrace{\ln(0,3)}_{\text{Carla: mal}} \right] \\
 &= -\frac{1}{3}(-0,105 - 0,223 - 1,204) = \mathbf{0,511}
 \end{aligned}$$



Ejemplo: calculemos el costo con 3 pacientes

Contexto: Un modelo predice si un paciente tiene una enfermedad ($y = 1$) o no ($y = 0$).

Nota: El logaritmo usado es el **logaritmo natural** ($\ln$, base $e \approx 2,718$), estándar en Deep Learning.

Paciente	Real (y)	Pred. ($\hat{y}$)	Costo individual
Ana	1 (enferma)	0.9	$-\ln(0,9) = 0,105$
Bruno	0 (sano)	0.2	$-\ln(0,8) = 0,223$
Carla	1 (enferma)	0.3	$-\ln(0,3) = 1,204$

Costo total:

$$\begin{aligned}\mathcal{L} &= -\frac{1}{3} \left[\underbrace{\ln(0,9)}_{\text{Ana: bien}} + \underbrace{\ln(0,8)}_{\text{Bruno: bien}} + \underbrace{\ln(0,3)}_{\text{Carla: mal}} \right] \\ &= -\frac{1}{3}(-0,105 - 0,223 - 1,204) = \mathbf{0,511}\end{aligned}$$

¿Qué dice este número?

- ▶ Ana: $\hat{y} = 0,9$ para $y = 1 \rightarrow$ **costo bajo** (buena predicción)
- ▶ Bruno: $\hat{y} = 0,2$ para $y = 0 \rightarrow$ **costo bajo** (acertó)
- ▶ Carla: $\hat{y} = 0,3$ para $y = 1 \rightarrow$ **costo alto** (falló, debió dar ≈ 1)

▷ Carla “arrastra” el costo hacia arriba. GD ajustará los pesos para que $\hat{y}$ de Carla suba.

¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ Clasificación binaria (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo

NO usar Cross-Entropy cuando:



¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**

NO usar Cross-Entropy cuando:



¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**
- ▶ Las etiquetas son $y \in \{0, 1\}$

NO usar Cross-Entropy cuando:



¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**
- ▶ Las etiquetas son $y \in \{0, 1\}$

▶ Es exactamente el caso de nuestra práctica: clasificar puntos en **2 clases** (lunas).

NO usar Cross-Entropy cuando:



¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**
- ▶ Las etiquetas son $y \in \{0, 1\}$

▷ Es exactamente el caso de nuestra práctica: clasificar puntos en **2 clases** (lunas).

NO usar Cross-Entropy cuando:

- ▶ La salida es un **número continuo** (regresión):
 - ▶ Predecir precio de una casa
 - ▶ Predecir temperatura
 - ▶ Predecir tiempo de entrega



Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**
- ▶ Las etiquetas son $y \in \{0, 1\}$

► Es exactamente el caso de nuestra práctica: clasificar puntos en **2 clases** (lunas).

NO usar Cross-Entropy cuando:

- ▶ La salida es un **número continuo** (regresión):
 - ▶ Predecir precio de una casa
 - ▶ Predecir temperatura
 - ▶ Predecir tiempo de entrega
- ▶ En ese caso usar **Error Cuadrático Medio** (MSE):

$$J = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$



¿Cuándo usar Cross-Entropy y cuándo no?

Sí usar Binary Cross-Entropy:

- ▶ **Clasificación binaria** (2 clases)
 - ▶ Spam o no spam
 - ▶ Enfermo o sano
 - ▶ Fraude o legítimo
- ▶ La salida es una **probabilidad** ($\hat{y} \in (0, 1)$) gracias a la **sigmoide**
- ▶ Las etiquetas son $y \in \{0, 1\}$

▷ Es exactamente el caso de nuestra práctica: clasificar puntos en **2 clases** (lunas).

NO usar Cross-Entropy cuando:

- ▶ La salida es un **número continuo** (regresión):
 - ▶ Predecir precio de una casa
 - ▶ Predecir temperatura
 - ▶ Predecir tiempo de entrega
- ▶ En ese caso usar **Error Cuadrático Medio** (MSE):

$$J = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

▷ Es exactamente el caso de nuestra práctica: clasificar puntos en **2 clases** (lunas)



Resumen: ¿qué función de costo usar?

Tarea	Función de costo	Ejemplo
Clasificación binaria	Binary Cross-Entropy (BCE)	Spam / no spam
Clasificación multiclase	Cross-Entropy categórica	Dígitos 0–9
Regresión	Error Cuadrático Medio (MSE)	Predecir precio



Resumen: ¿qué función de costo usar?

Tarea	Función de costo	Ejemplo
Clasificación binaria	Binary Cross-Entropy (BCE)	Spam / no spam
Clasificación multiclase	Cross-Entropy categórica	Dígitos 0–9
Regresión	Error Cuadrático Medio (MSE)	Predecir precio

▷ En nuestra práctica usaremos **BCE** porque clasificamos puntos en 2 clases (lunas).



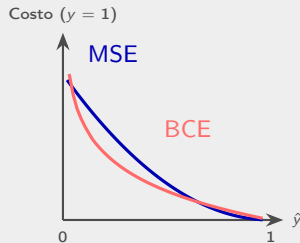
¿Por qué no usar MSE para clasificación?

- ▶ Si $y = 1$ y $\hat{y} = 0,01$:
MSE: $(1 - 0,01)^2 = 0,98$
BCE: $-\ln(0,01) = 4,60$



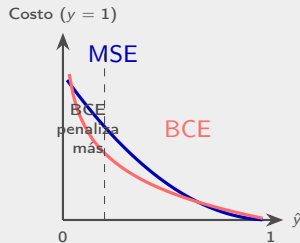
¿Por qué no usar MSE para clasificación?

- ▶ Si $y = 1$ y $\hat{y} = 0,01$:
MSE: $(1 - 0,01)^2 = 0,98$
BCE: $-\ln(0,01) = 4,60$
- ▶ BCE **penaliza mucho más** las predicciones equivocadas



¿Por qué no usar MSE para clasificación?

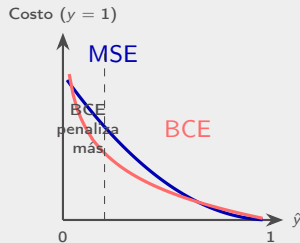
- ▶ Si $y = 1$ y $\hat{y} = 0,01$:
MSE: $(1 - 0,01)^2 = 0,98$
BCE: $-\ln(0,01) = 4,60$
- ▶ BCE **penaliza mucho más** las predicciones equivocadas
- ▶ Con MSE, el gradiente se vuelve **muy pequeño** cerca de 0 y 1 (por la sigmoide)
→ aprendizaje lento



¿Por qué no usar MSE para clasificación?

- ▶ Si $y = 1$ y $\hat{y} = 0,01$:
MSE: $(1 - 0,01)^2 = 0,98$
BCE: $-\ln(0,01) = 4,60$
- ▶ BCE **penaliza mucho más** las predicciones equivocadas
- ▶ Con MSE, el gradiente se vuelve **muy pequeño** cerca de 0 y 1 (por la sigmoide)
→ aprendizaje lento

▷ BCE genera gradientes más **útiles** → GD aprende más rápido.



¿Qué es el gradiente?

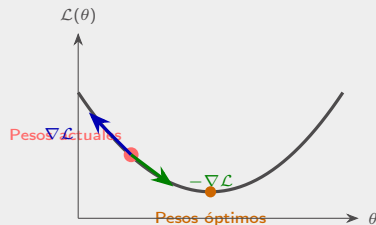
El **gradiente** $\nabla \mathcal{L}(\theta)$ es un vector que indica:

- La **dirección** en la que el costo **crece más rápido**



El **gradiente** $\nabla \mathcal{L}(\theta)$ es un vector que indica:

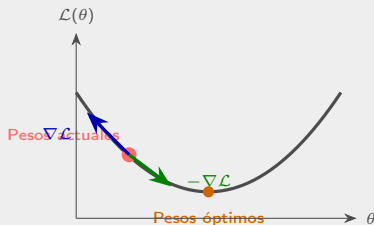
- La **dirección** en la que el costo **crece más rápido**
- La **magnitud**: qué tan empinada es la pendiente



El **gradiente** $\nabla \mathcal{L}(\theta)$ es un vector que indica:

- ▶ La **dirección** en la que el costo **crece más rápido**
- ▶ La **magnitud**: qué tan empinada es la pendiente
- ▶ Si vamos en dirección **contraria** ($-\nabla \mathcal{L}$), el costo **disminuye**

▷ Gradient Descent sigue la dirección $-\nabla \mathcal{L}$ para **reducir el error** paso a paso.



Gradientes en nuestra red neuronal

En nuestra red, necesitamos el gradiente del costo **respecto a cada parámetro**:

$$\frac{\partial \mathcal{L}}{\partial W_1}, \quad \frac{\partial \mathcal{L}}{\partial b_1}, \quad \frac{\partial \mathcal{L}}{\partial W_2}, \quad \frac{\partial \mathcal{L}}{\partial b_2}$$



Gradientes en nuestra red neuronal

En nuestra red, necesitamos el gradiente del costo **respecto a cada parámetro**:

$$\frac{\partial \mathcal{L}}{\partial W_1}, \quad \frac{\partial \mathcal{L}}{\partial b_1}, \quad \frac{\partial \mathcal{L}}{\partial W_2}, \quad \frac{\partial \mathcal{L}}{\partial b_2}$$

- ▶ Cada derivada parcial indica **cuánto** y en **qué dirección** cambiar ese peso para reducir el error
- ▶ Si $\frac{\partial \mathcal{L}}{\partial W} > 0$: el peso está “demasiado alto” → hay que **reducirlo**
- ▶ Si $\frac{\partial \mathcal{L}}{\partial W} < 0$: el peso está “demasiado bajo” → hay que **aumentarlo**



Gradientes en nuestra red neuronal

En nuestra red, necesitamos el gradiente del costo **respecto a cada parámetro**:

$$\frac{\partial \mathcal{L}}{\partial W_1}, \quad \frac{\partial \mathcal{L}}{\partial b_1}, \quad \frac{\partial \mathcal{L}}{\partial W_2}, \quad \frac{\partial \mathcal{L}}{\partial b_2}$$

- ▶ Cada derivada parcial indica **cuánto** y en **qué dirección** cambiar ese peso para reducir el error
 - ▶ Si $\frac{\partial \mathcal{L}}{\partial W} > 0$: el peso está “demasiado alto” → hay que **reducirlo**
 - ▶ Si $\frac{\partial \mathcal{L}}{\partial W} < 0$: el peso está “demasiado bajo” → hay que **aumentarlo**
- ▶ Calcular estos gradientes es el trabajo de **Backpropagation** (lo veremos más adelante).



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$

- θ_{actual} : los pesos y sesgos actuales (W_1, b_1, W_2, b_2)



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$

- ▶ θ_{actual} : los pesos y sesgos actuales (W_1, b_1, W_2, b_2)
- ▶ $\nabla \mathcal{L}(\theta)$: el gradiente (indica hacia dónde crece el error)



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$

- ▶ θ_{actual} : los pesos y sesgos actuales (W_1, b_1, W_2, b_2)
- ▶ $\nabla \mathcal{L}(\theta)$: el gradiente (indica hacia dónde crece el error)
- ▶ α : la **tasa de aprendizaje** (*learning rate*)



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$

- ▶ θ_{actual} : los pesos y sesgos actuales (W_1, b_1, W_2, b_2)
- ▶ $\nabla \mathcal{L}(\theta)$: el gradiente (indica hacia dónde crece el error)
- ▶ α : la **tasa de aprendizaje** (*learning rate*)
- ▶ El signo **negativo** hace que vayamos **cuesta abajo**



La regla de actualización

En cada paso, actualizamos los parámetros así:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot \nabla \mathcal{L}(\theta_{\text{actual}})$$

- ▶ θ_{actual} : los pesos y sesgos actuales (W_1, b_1, W_2, b_2)
- ▶ $\nabla \mathcal{L}(\theta)$: el gradiente (indica hacia dónde crece el error)
- ▶ α : la **tasa de aprendizaje** (*learning rate*)
- ▶ El signo **negativo** hace que vayamos **cuesta abajo**

► En la práctica esto se aplica a **cada parámetro** individualmente:

$$W_1 \leftarrow W_1 - \alpha \cdot dW_1, \quad b_1 \leftarrow b_1 - \alpha \cdot db_1, \quad \text{etc.}$$



¿Qué pasa con la tasa de aprendizaje α ?

$$\alpha = 0,01$$



Convergencia muy lenta

$$\alpha = 0,5$$



Convergencia eficiente

$$\alpha = 2,0$$



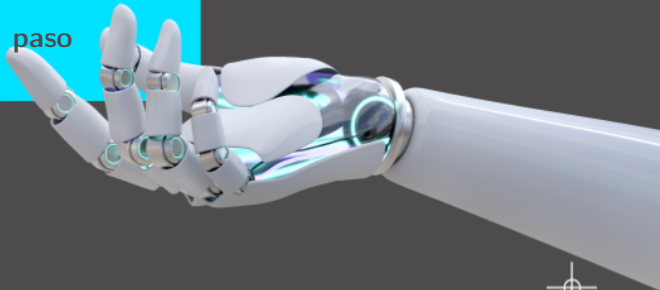
No converge (rebota)

En la práctica experimentarán con $\alpha \in \{0,01, 0,1, 0,5, 2,0\}$ y compararán las curvas de pérdida.



Desarrollo

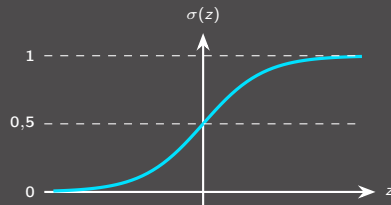
Construyendo la red neuronal paso a paso



Funciones de activación: Sigmoides

La **sigmoide** transforma cualquier número en una probabilidad entre 0 y 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



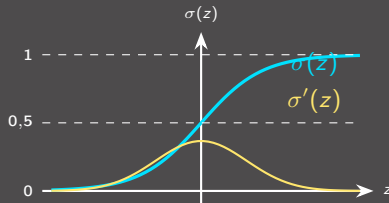
Funciones de activación: Sigmoides

La **sigmoide** transforma cualquier número en una probabilidad entre 0 y 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Su **derivada** (necesaria para backpropagation):

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$



Funciones de activación: Sigmoide

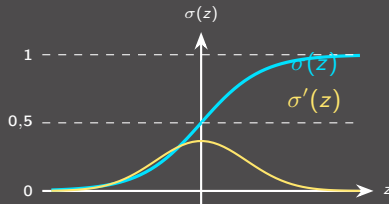
La **sigmoide** transforma cualquier número en una probabilidad entre 0 y 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Su **derivada** (necesaria para backpropagation):

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

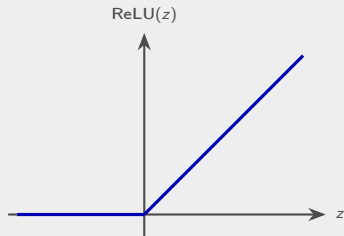
Uso en nuestra red: en la **capa de salida** para obtener la probabilidad de pertenecer a la clase 1.



Funciones de activación: ReLU

ReLU (Rectified Linear Unit) es la activación más usada en capas ocultas:

$$\text{ReLU}(z) = \max(0, z)$$



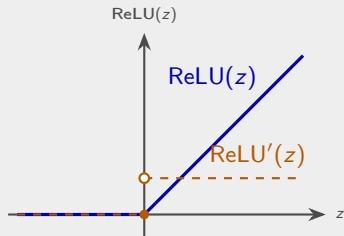
Funciones de activación: ReLU

ReLU (Rectified Linear Unit) es la activación más usada en capas ocultas:

$$\text{ReLU}(z) = \max(0, z)$$

Su **derivada**:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$



Funciones de activación: ReLU

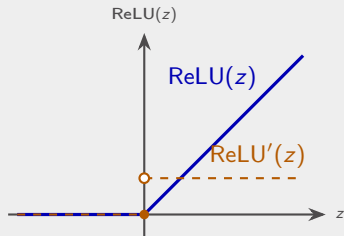
ReLU (Rectified Linear Unit) es la activación más usada en capas ocultas:

$$\text{ReLU}(z) = \max(0, z)$$

Su **derivada**:

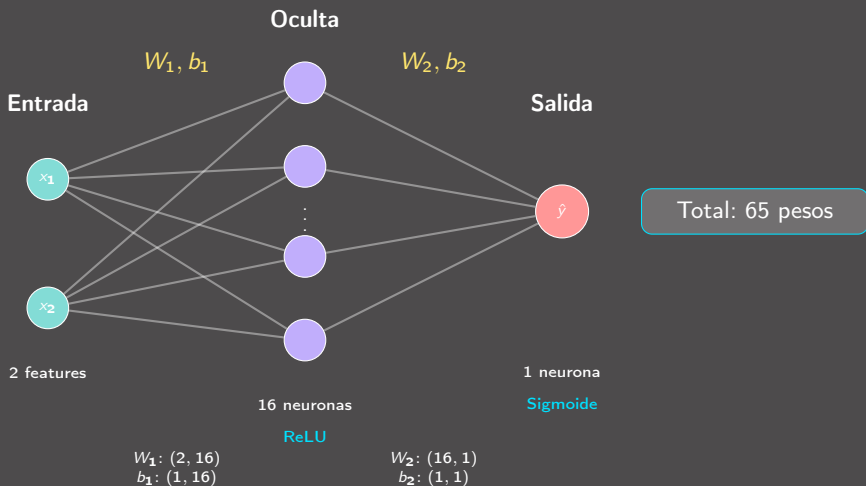
$$\text{ReLU}'(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$

Uso en nuestra red: en la **capa oculta** (16 neuronas). Introduce **no linealidad**, lo que permite separar las lunas.



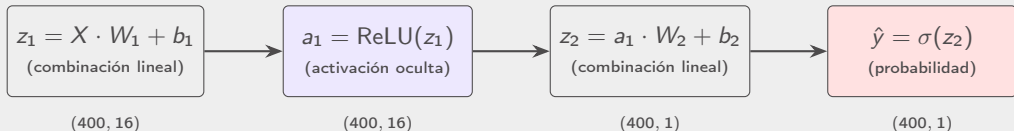
Sin ReLU, la red sería solo una transformación lineal

Arquitectura de nuestra red neuronal



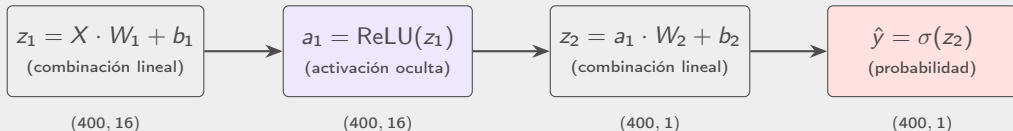
Forward Pass: propagación hacia adelante

Los datos fluyen de izquierda a derecha por la red:



Forward Pass: propagación hacia adelante

Los datos fluyen de izquierda a derecha por la red:



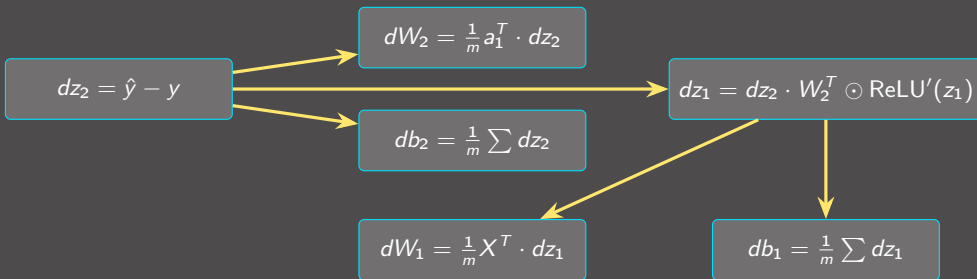
- X : nuestros 400 puntos con 2 features (400×2)
- $\hat{y} \in (0, 1)$: probabilidad de que cada punto sea clase 1
- Guardamos $z_1, a_1, z_2, \hat{y}$ en **cache** (los necesitamos para backprop)



Backpropagation: calculando los gradientes

Propagamos el error **hacia atrás** usando la regla de la cadena:

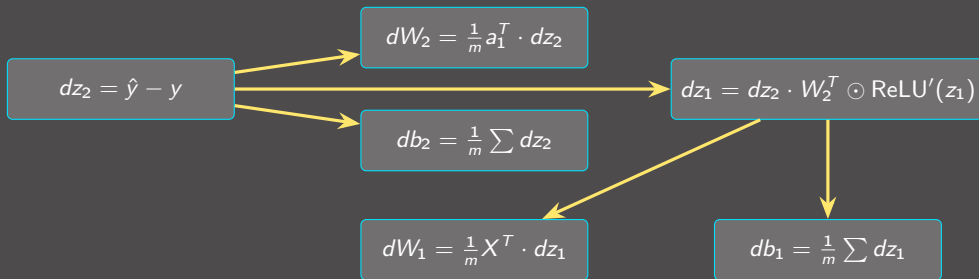
← **Dirección del cálculo (atrás)**



Backpropagation: calculando los gradientes

Propagamos el error **hacia atrás** usando la regla de la cadena:

← **Dirección del cálculo (atrás)**



▷ El símbolo $\odot$ significa multiplicación **elemento a elemento**.

▷ Cada dW y db nos dice **cuánto y en qué dirección** ajustar cada peso.



Ejemplo paso a paso: dataset y red $2 \rightarrow 2 \rightarrow 1$

Simplificamos a 2 neuronas ocultas para calcular a mano. 10 puntos 2D:

i	x_1	x_2	y
1	1	2	1
2	2	1	1
3	1.5	1.5	1
4	2	2	1
5	1	1	1
6	-1	-2	0
7	-2	-1	0
8	-1,5	-1,5	0
9	-2	-2	0
10	-1	-1	0

Pesos iniciales (valores pequeños):

$$W_1 = \begin{pmatrix} 0,5 & -0,3 \\ 0,2 & 0,4 \end{pmatrix} \quad b_1 = \begin{pmatrix} 0 & 0 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0,6 \\ -0,5 \end{pmatrix} \quad b_2 = 0$$



Ejemplo paso a paso: dataset y red $2 \rightarrow 2 \rightarrow 1$

Simplificamos a 2 neuronas ocultas para calcular a mano. 10 puntos 2D:

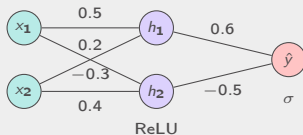
i	x_1	x_2	y
1	1	2	1
2	2	1	1
3	1.5	1.5	1
4	2	2	1
5	1	1	1
6	-1	-2	0
7	-2	-1	0
8	-1.5	-1.5	0
9	-2	-2	0
10	-1	-1	0

Pesos iniciales (valores pequeños):

$$W_1 = \begin{pmatrix} 0,5 & -0,3 \\ 0,2 & 0,4 \end{pmatrix} \quad b_1 = \begin{pmatrix} 0 & 0 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0,6 \\ -0,5 \end{pmatrix} \quad b_2 = 0$$

Arquitectura:



Ejemplo: Forward — Paso 1: z_1 y a_1 (clase 1)

Notación matricial: $z_1 = X \cdot W_1 + b_1$ **Notación iterativa:** $z_{1,j}^{(i)} = \sum_{k=1}^2 x_k^{(i)} \cdot W_{1,kj} + b_{1,j}$

Detalle para muestra $i = 1$ ($x_1 = 1$, $x_2 = 2$, clase 1):

$$z_{1,1}^{(1)} = x_1^{(1)} \cdot W_{1,11} + x_2^{(1)} \cdot W_{1,21} = 1(0,5) + 2(0,2) = 0,9$$

$$z_{1,2}^{(1)} = x_1^{(1)} \cdot W_{1,12} + x_2^{(1)} \cdot W_{1,22} = 1(-0,3) + 2(0,4) = 0,5$$



Ejemplo: Forward — Paso 1: z_1 y a_1 (clase 1)

Notación matricial: $z_1 = X \cdot W_1 + b_1$ **Notación iterativa:** $z_{1,j}^{(i)} = \sum_{k=1}^2 x_k^{(i)} \cdot W_{1,kj} + b_{1,j}$

Detalle para muestra $i = 1$ ($x_1 = 1$, $x_2 = 2$, clase 1):

$$z_{1,1}^{(1)} = x_1^{(1)} \cdot W_{1,11} + x_2^{(1)} \cdot W_{1,21} = 1(0,5) + 2(0,2) = 0,9$$

$$z_{1,2}^{(1)} = x_1^{(1)} \cdot W_{1,12} + x_2^{(1)} \cdot W_{1,22} = 1(-0,3) + 2(0,4) = 0,5$$

Activación ReLU:

$$a_1^{(1)} = \text{ReLU}([0,9, 0,5]) = [0,9, 0,5]$$

Ambos valores son positivos $\rightarrow$ ReLU **no los modifica**.



Ejemplo: Forward — Paso 1: z_1 y a_1 (clase 0)

Detalle para muestra $i = 6$ ($x_1 = -1$, $x_2 = -2$, clase 0):

$$z_{1,1}^{(6)} = (-1)(0,5) + (-2)(0,2) = -0,9$$

$$z_{1,2}^{(6)} = (-1)(-0,3) + (-2)(0,4) = -0,5$$



Ejemplo: Forward — Paso 1: z_1 y a_1 (clase 0)

Detalle para muestra $i = 6$ ($x_1 = -1$, $x_2 = -2$, clase 0):

$$z_{1,1}^{(6)} = (-1)(0,5) + (-2)(0,2) = -0,9$$

$$z_{1,2}^{(6)} = (-1)(-0,3) + (-2)(0,4) = -0,5$$

Activación ReLU:

$$a_1^{(6)} = \text{ReLU}([-0,9, -0,5]) = [0, 0]$$

Ambos valores son negativos $\rightarrow$ ReLU los **corta a cero**.

► Esto es clave: las muestras de clase 0 tienen activaciones nulas, lo que afectará el backpropagation (sus gradientes serán cero).



Ejemplo: Forward — Paso 2: z_2 , $\hat{y}$ y resultados

Matricial: $z_2 = a_1 \cdot W_2 + b_2$ **Iterativa:** $z_2^{(i)} = \sum_{j=1}^2 a_{1,j}^{(i)} \cdot W_{2,j} + b_2$ $\hat{y}^{(i)} = \sigma(z_2^{(i)})$

Muestra 1:

$$z_2^{(1)} = 0,9(0,6) + 0,5(-0,5) = 0,29$$

$$\hat{y}^{(1)} = \sigma(0,29) = 0,572$$

Muestra 6:

$$z_2^{(6)} = 0(0,6) + 0(-0,5) = 0$$

$$\hat{y}^{(6)} = \sigma(0) = 0,500$$



Ejemplo: Forward — Paso 2: z_2 , $\hat{y}$ y resultados

Matricial: $z_2 = a_1 \cdot W_2 + b_2$ **Iterativa:** $z_2^{(i)} = \sum_{j=1}^2 a_{1,j}^{(i)} \cdot W_{2,j} + b_2$ $\hat{y}^{(i)} = \sigma(z_2^{(i)})$

Muestra 1:

$$z_2^{(1)} = 0,9(0,6) + 0,5(-0,5) = 0,29$$

$$\hat{y}^{(1)} = \sigma(0,29) = 0,572$$

Muestra 6:

$$z_2^{(6)} = 0(0,6) + 0(-0,5) = 0$$

$$\hat{y}^{(6)} = \sigma(0) = 0,500$$

i	$a_{1,1}$	$a_{1,2}$	z_2	$\hat{y}$	y
1	0.9	0.5	0.29	0.572	1
2	1.2	0	0.72	0.673	1
3	1.05	0.15	0.555	0.635	1
4	1.4	0.2	0.74	0.677	1
5	0.7	0.1	0.37	0.591	1
6	0	0	0	0.500	0
7	0	0.2	-0.1	0.475	0
8	0	0	0	0.500	0
9	0	0	0	0.500	0
10	0	0	0	0.500	0

Predicciones mediocres: clase 1 $\approx 0,6$, clase 0 $\approx 0,5$. El modelo aún no ha aprendido.

Ejemplo: cálculo de la pérdida BCE

Matricial: $\mathcal{L} = -\frac{1}{m} \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ **Iterativa:**

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Para las 10 muestras:

$$\begin{aligned} \mathcal{L} = -\frac{1}{10} & \left[\underbrace{\log(0,572) + \log(0,673) + \log(0,635) + \log(0,677) + \log(0,591)}_{\text{clase 1: deberían ser } \approx 1} \right. \\ & \left. + \underbrace{\log(0,500) + \log(0,525) + \log(0,500) + \log(0,500) + \log(0,500)}_{\text{clase 0: deberían ser } \approx 0, \text{ usamos } \log(1 - \hat{y})} \right] \end{aligned}$$



Ejemplo: cálculo de la pérdida BCE

Matricial: $\mathcal{L} = -\frac{1}{m} \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ **Iterativa:**

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Para las 10 muestras:

$$\begin{aligned} \mathcal{L} = & -\frac{1}{10} \left[\underbrace{\log(0,572) + \log(0,673) + \log(0,635) + \log(0,677) + \log(0,591)}_{\text{clase 1: deberían ser } \approx 1} \right. \\ & \left. + \underbrace{\log(0,500) + \log(0,525) + \log(0,500) + \log(0,500) + \log(0,500)}_{\text{clase 0: deberían ser } \approx 0, \text{ usamos } \log(1-\hat{y})} \right] \end{aligned}$$

$$\mathcal{L} = -\frac{1}{10} (-0,558 - 0,396 - 0,454 - 0,390 - 0,526 - 0,693 - 0,654 - 0,693 - 0,693 - 0,693)$$

$$\mathcal{L} = -\frac{1}{10} (-5,750) = 0,575$$

Valor cercano a $\log(2) = 0,693$ (aleatorio). GD debe **reducir** este número.



Ejemplo: Backpropagation — gradientes paso a paso

Paso 1: $dz_2^{(i)} = \hat{y}^{(i)} - y^{(i)}$ (error por muestra)

i	1	2	3	4	5	6	7	8	9	10
$dz_2^{(i)}$	-0,43	-0,33	-0,37	-0,32	-0,41	0,50	0,48	0,50	0,50	0,50



Ejemplo: Backpropagation — gradientes paso a paso

Paso 1: $dz_2^{(i)} = \hat{y}^{(i)} - y^{(i)}$ (error por muestra)

i	1	2	3	4	5	6	7	8	9	10
$dz_2^{(i)}$	-0,43	-0,33	-0,37	-0,32	-0,41	0,50	0,48	0,50	0,50	0,50

Paso 2: dW_2 Matricial: $dW_2 = \frac{1}{m} a_1^T \cdot dz_2$ Iterativa: $dW_{2,j} = \frac{1}{m} \sum_{i=1}^m a_{1,j}^{(i)} \cdot dz_2^{(i)}$

$$dW_{2,1} = \frac{1}{10} [0,9(-0,43) + 1,2(-0,33) + \dots + 0(0,50)] = -0,143$$

$$dW_{2,2} = \frac{1}{10} [0,5(-0,43) + 0(-0,33) + \dots + 0(0,50)] = -0,022$$



Ejemplo: Backpropagation — gradientes paso a paso

Paso 1: $dz_2^{(i)} = \hat{y}^{(i)} - y^{(i)}$ (error por muestra)

i	1	2	3	4	5	6	7	8	9	10
$dz_2^{(i)}$	-0,43	-0,33	-0,37	-0,32	-0,41	0,50	0,48	0,50	0,50	0,50

Paso 2: dW_2 Matricial: $dW_2 = \frac{1}{m} a_1^T \cdot dz_2$ Iterativa: $dW_{2,j} = \frac{1}{m} \sum_{i=1}^m a_{1,j}^{(i)} \cdot dz_2^{(i)}$

$$dW_{2,1} = \frac{1}{10} [0,9(-0,43) + 1,2(-0,33) + \dots + 0(0,50)] = -0,143$$

$$dW_{2,2} = \frac{1}{10} [0,5(-0,43) + 0(-0,33) + \dots + 0(0,50)] = -0,022$$

Paso 3: db_2 $db_2 = \frac{1}{m} \sum_{i=1}^m dz_2^{(i)} = \frac{1}{10} (-0,43 - 0,33 \dots + 0,50) = 0,012$



Ejemplo: Backpropagation — capa oculta

Paso 4: dz_1 Matricial: $dz_1 = dz_2 \cdot W_2^T \odot \text{ReLU}'(z_1)$

Iterativa: $dz_{1,j}^{(i)} = dz_2^{(i)} \cdot W_{2,j} \cdot \mathbb{I}[z_{1,j}^{(i)} > 0]$

Muestra 1: $dz_{1,1}^{(1)} = (-0,43)(0,6) \cdot 1 = -0,258$, $dz_{1,2}^{(1)} = (-0,43)(-0,5) \cdot 1 = 0,215$

Muestra 6: $dz_{1,1}^{(6)} = (0,50)(0,6) \cdot 0 = 0$, $dz_{1,2}^{(6)} = (0,50)(-0,5) \cdot 0 = 0$ (ReLU bloquea)



Ejemplo: Backpropagation — capa oculta

Paso 4: dz_1 Matricial: $dz_1 = dz_2 \cdot W_2^T \odot \text{ReLU}'(z_1)$

Iterativa: $dz_{1,j}^{(i)} = dz_2^{(i)} \cdot W_{2,j} \cdot \mathbb{I}[z_{1,j}^{(i)} > 0]$

Muestra 1: $dz_{1,1}^{(1)} = (-0,43)(0,6) \cdot 1 = -0,258$, $dz_{1,2}^{(1)} = (-0,43)(-0,5) \cdot 1 = 0,215$

Muestra 6: $dz_{1,1}^{(6)} = (0,50)(0,6) \cdot 0 = 0$, $dz_{1,2}^{(6)} = (0,50)(-0,5) \cdot 0 = 0$ (ReLU bloquea)

Paso 5: dW_1 Matricial: $dW_1 = \frac{1}{m} X^T \cdot dz_1$ Iterativa: $dW_{1,kj} = \frac{1}{m} \sum_{i=1}^m x_k^{(i)} \cdot dz_{1,j}^{(i)}$

Solo las muestras de clase 1 contribuyen (clase 0: $dz_1 = 0$ por ReLU).

$$dW_1 \approx \begin{pmatrix} -0,061 & 0,051 \\ -0,058 & 0,049 \end{pmatrix}, \quad db_1 \approx \begin{pmatrix} -0,038 & 0,032 \end{pmatrix}$$



Ejemplo: Backpropagation — capa oculta

Paso 4: dz_1 Matricial: $dz_1 = dz_2 \cdot W_2^T \odot \text{ReLU}'(z_1)$

Iterativa: $dz_{1,j}^{(i)} = dz_2^{(i)} \cdot W_{2,j} \cdot \mathbb{I}[z_{1,j}^{(i)} > 0]$

Muestra 1: $dz_{1,1}^{(1)} = (-0,43)(0,6) \cdot 1 = -0,258$, $dz_{1,2}^{(1)} = (-0,43)(-0,5) \cdot 1 = 0,215$

Muestra 6: $dz_{1,1}^{(6)} = (0,50)(0,6) \cdot 0 = 0$, $dz_{1,2}^{(6)} = (0,50)(-0,5) \cdot 0 = 0$ (ReLU bloquea)

Paso 5: dW_1 Matricial: $dW_1 = \frac{1}{m} X^T \cdot dz_1$ Iterativa: $dW_{1,kj} = \frac{1}{m} \sum_{i=1}^m x_k^{(i)} \cdot dz_{1,j}^{(i)}$

Solo las muestras de clase 1 contribuyen (clase 0: $dz_1 = 0$ por ReLU).

$$dW_1 \approx \begin{pmatrix} -0,061 & 0,051 \\ -0,058 & 0,049 \end{pmatrix}, \quad db_1 \approx \begin{pmatrix} -0,038 & 0,032 \end{pmatrix}$$

► Los gradientes nos dicen: **aumentar** $W_{2,1}$ (es negativo) y **reducir** b_2 (es positivo).



Ejemplo: actualización y clasificación final

Regla de GD con $\alpha = 0,5$: $\theta \leftarrow \theta - 0,5 \cdot \nabla \theta$

Actualización de W_2 :

$$W_{2,1} = 0,6 - 0,5(-0,143) = 0,672$$

$$W_{2,2} = -0,5 - 0,5(-0,022) = -0,489$$

$$b_2 = 0 - 0,5(0,012) = -0,006$$



Ejemplo: actualización y clasificación final

Regla de GD con $\alpha = 0,5$: $\theta \leftarrow \theta - 0,5 \cdot \nabla \theta$

Actualización de W_2 :

$$W_{2,1} = 0,6 - 0,5(-0,143) = 0,672$$

$$W_{2,2} = -0,5 - 0,5(-0,022) = -0,489$$

$$b_2 = 0 - 0,5(0,012) = -0,006$$

Actualización de W_1 :

$$W'_1 \approx \begin{pmatrix} 0,531 & -0,326 \\ 0,229 & 0,376 \end{pmatrix}$$

$$b'_1 \approx (0,019 \quad -0,016)$$



Ejemplo: actualización y clasificación final

Regla de GD con $\alpha = 0,5$: $\theta \leftarrow \theta - 0,5 \cdot \nabla \theta$

Actualización de W_2 :

$$W_{2,1} = 0,6 - 0,5(-0,143) = 0,672$$

$$W_{2,2} = -0,5 - 0,5(-0,022) = -0,489$$

$$b_2 = 0 - 0,5(0,012) = -0,006$$

Actualización de W_1 :

$$W'_1 \approx \begin{pmatrix} 0,531 & -0,326 \\ 0,229 & 0,376 \end{pmatrix}$$

$$b'_1 \approx (0,019 \quad -0,016)$$

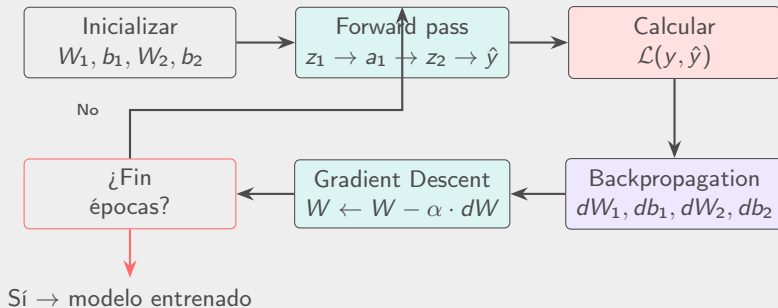
Clasificación ($\hat{y} > 0,5 \rightarrow$ clase 1):

i	$\hat{y}$ época 0	$\hat{y}$ época 1	Clase
1	0.572	0.594 ↑	1 ✓
2	0.673	0.702 ↑	1 ✓
5	0.591	0.612 ↑	1 ✓
6	0.500	0.497 ↓	0 ✓
7	0.475	0.471 ↓	0 ✓
10	0.500	0.497 ↓	0 ✓

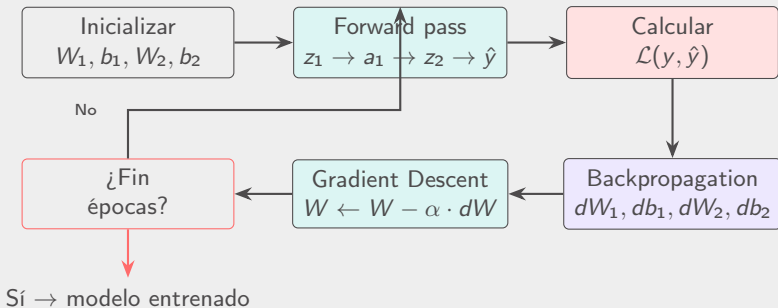
▷ Tras 1 sola época, las predicciones ya mejoran. Después de cientos de épocas, $\hat{y}$ se acercará a 0 o 1.



El ciclo completo de entrenamiento



El ciclo completo de entrenamiento



En la práctica: este ciclo se repite durante **2000 épocas**. En cada época se procesan los 400 datos completos (batch gradient descent).



Épocas y convergencia

- ▶ **Época:** una pasada completa por todos los datos de entrenamiento



Épocas y convergencia

- ▶ **Época:** una pasada completa por todos los datos de entrenamiento
- ▶ En cada época: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update



Épocas y convergencia

- ▶ **Época**: una pasada completa por todos los datos de entrenamiento
- ▶ En cada época: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update
- ▶ El costo debe **disminuir** época tras época



Épocas y convergencia

- ▶ **Época**: una pasada completa por todos los datos de entrenamiento
- ▶ En cada época: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update
- ▶ El costo debe **disminuir** época tras época

En nuestra práctica:

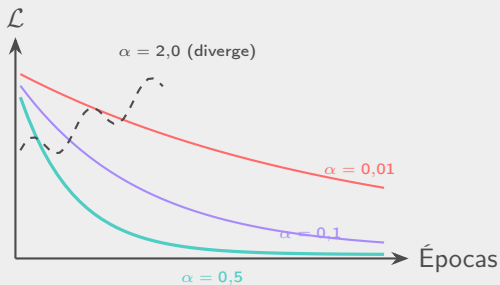
400 datos, 2000 épocas, $\alpha = 0,5$

La pérdida inicial debe estar cerca de $\log(2) \approx 0,6931$ (predicción aleatoria al 50 %).



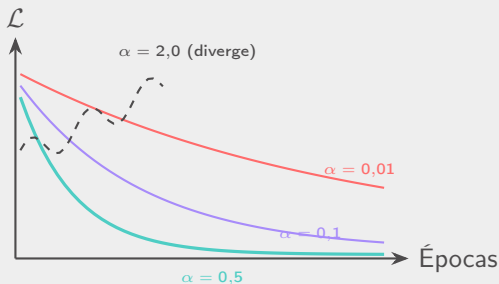
Experimentación: comparar learning rates

En la práctica probarán $\alpha \in \{0,01, 0,1, 0,5, 2,0\}$. ¿Qué esperar?



Experimentación: comparar learning rates

En la práctica probarán $\alpha \in \{0,01, 0,1, 0,5, 2,0\}$. ¿Qué esperar?



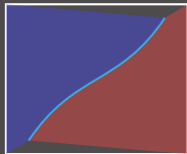
- ▶ α pequeño: converge pero muy lento (puede necesitar más épocas)
- ▶ α grande: la pérdida **oscila** o **diverge** (nunca aprende)
- ▶ α adecuado: convergencia rápida y estable



Experimentación: tamaño de la capa oculta

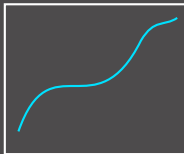
En la práctica probarán capas ocultas de $\{2, 4, 16, 64\}$ neuronas:

$$n_h = 2$$



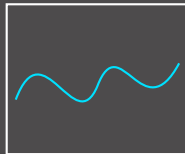
Muy simple

$$n_h = 4$$



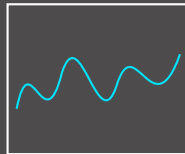
Mejor, pero limitado

$$n_h = 16$$



Buena separación

$$n_h = 64$$

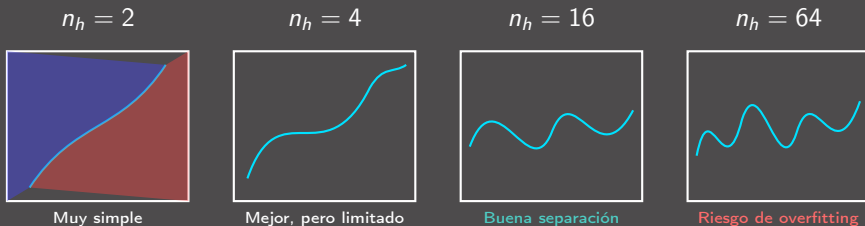


Riesgo de overfitting



Experimentación: tamaño de la capa oculta

En la práctica probarán capas ocultas de $\{2, 4, 16, 64\}$ neuronas:



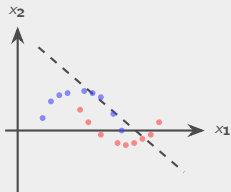
- ▶ Pocas neuronas → **underfitting**: la red no puede capturar la forma de las lunas
- ▶ Muchas neuronas → **overfitting**: la frontera se ajusta al ruido de los datos
- ▶ 16 neuronas es un buen equilibrio para nuestro dataset



Resultado: superficie de decisión

Antes del entrenamiento:

Pesos aleatorios $\rightarrow$ frontera sin sentido.



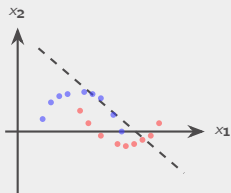
Precisión $\approx 50\%$



Resultado: superficie de decisión

Antes del entrenamiento:

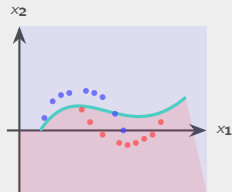
Pesos aleatorios $\rightarrow$ frontera sin sentido.



Precisión $\approx 50\%$

Después de 2000 épocas:

GD encontró pesos óptimos $\rightarrow$ frontera curva.

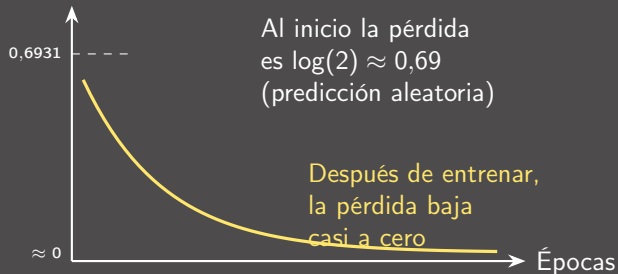


Precisión $> 95\%$



Curva de aprendizaje típica

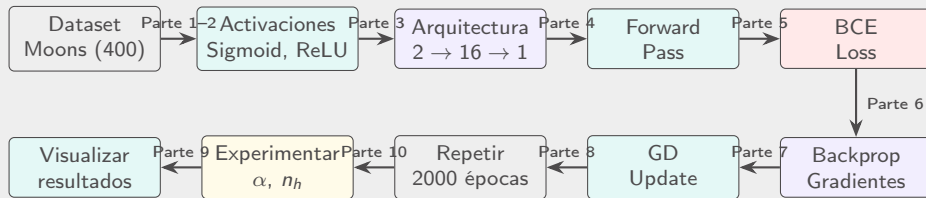
Binary Cross-Entropy $\mathcal{L}$



En la práctica imprimirán la pérdida cada 200 épocas para verificar que baja.



Resumen: todo el flujo en una imagen



Conclusiones

Resumen y práctica



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$
3. **Binary Cross-Entropy** mide el error de clasificación



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$
3. **Binary Cross-Entropy** mide el error de clasificación
4. **Backpropagation** calcula los gradientes dW_1, db_1, dW_2, db_2



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$
3. **Binary Cross-Entropy** mide el error de clasificación
4. **Backpropagation** calcula los gradientes dW_1, db_1, dW_2, db_2
5. **Gradient Descent** actualiza: $W \leftarrow W - \alpha \cdot dW$



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$
3. **Binary Cross-Entropy** mide el error de clasificación
4. **Backpropagation** calcula los gradientes dW_1, db_1, dW_2, db_2
5. **Gradient Descent** actualiza: $W \leftarrow W - \alpha \cdot dW$
6. El **learning rate** α y el **número de neuronas** impactan directamente en el resultado



Puntos clave para recordar

1. **Sigmoide** convierte valores en probabilidades (salida); **ReLU** introduce no linealidad (capa oculta)
2. El **forward pass** propaga datos: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y}$
3. **Binary Cross-Entropy** mide el error de clasificación
4. **Backpropagation** calcula los gradientes dW_1, db_1, dW_2, db_2
5. **Gradient Descent** actualiza: $W \leftarrow W - \alpha \cdot dW$
6. El **learning rate** α y el **número de neuronas** impactan directamente en el resultado

Sin Gradient Descent, la red neuronal no podría aprender a separar las lunas. 

Actividad práctica: Gradient Descent desde cero

Práctica en Jupyter Notebook

Instrucciones:

1. Trabajar en **parejas**
2. Abrir `gd_practica.ipynb`
3. Completar las **10 tareas**:
 - T1–T2** Sigmoid, ReLU y sus derivadas
 - T3** Inicializar pesos y sesgos
 - T4** Forward pass
 - T5** Binary Cross-Entropy
 - T6** Backpropagation
 - T7** Actualización de parámetros (GD)
 - T8** Bucle de entrenamiento
 - T9–T10** Experimentar con α y n_h

Verificar en cada tarea:

- ▶ Ejecutar las celdas de **verificación** (no modificarlas)
- ▶ Comparar con los valores **esperados**

Resultado esperado:

- ▶ Curva de pérdida decreciente
- ▶ Precisión $> 95\%$
- ▶ Superficie de decisión que separe las lunas



Mapa de conceptos de la sesión

Concepto	Función e importancia	Conexión con Deep Learning
Sigmoid σ	Convierte valores reales a probabilidades $[0, 1]$. Salida del modelo.	Base de softmax (multiclase). Usada en LSTMs y modelos generativos.
ReLU	No linealidad en capas ocultas: $\max(0, z)$. Permite aprender patrones complejos.	Variantes: LeakyReLU, GELU (Transformers), Swish (EfficientNet).
Forward pass	Propaga entrada $\rightarrow$ predicción: $z_1, a_1, z_2, \hat{y}$.	Igual en CNNs, RNNs, Transformers. En inferencia solo se usa forward.
BCE Loss	Mide error de clasificación binaria. Guía la dirección del aprendizaje.	Se extiende a Cross-Entropy categórica (multiclase) y focal loss (desbalanceo).
Backpropagation	Calcula $\nabla \mathcal{L}$ con regla de la cadena para cada parámetro.	Misma idea en redes profundas. Problemas: vanishing/exploding gradients .
Gradient Descent	Actualiza pesos: $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$. Motor del aprendizaje.	Variantes: SGD , Adam , RMSprop . Adam es el estándar actual.



- ▶ Goodfellow, Bengio & Courville (2016). *Deep Learning*, Cap. 4 y 8.
- ▶ Kingma & Ba (2015). *Adam: A Method for Stochastic Optimization*.
- ▶ Ruder (2016). *An overview of gradient descent optimization algorithms*.





UTEC Posgrado



UTEC Posgrado